

Poetry — Dependency Management

How `hathor-core` declares what it needs, pins exactly what it resolved, and rebuilds the same environment everywhere — `pyproject.toml`, the lock file, and why not plain pip.

SUBJECT hathor-core · Part I · Track C (the stack)

CHAPTER 13 · Foundations · The Stack

BRANCH study/hathor-core-studies

EDITION 2026 · v1

Poetry · `pyproject.toml` · `poetry.lock` · Dependency resolution · Version constraints ·
Dependency groups · Build backend · PEP 517/518

Table of Contents

Chapter 13 — Poetry: Dependency Management	3
13.1 The problem: a list of packages is not enough	3
13.2 Two files: declare in pyproject.toml, pin in poetry.lock	4
13.3 Reading hathon-core's pyproject.toml	5
13.4 The build backend: pyproject.toml is also packaging config	6
13.5 Why Poetry, and not the alternatives?	6
13.6 Bridge — Poetry across the project	7
Recap	8

Chapter 13 — Poetry: Dependency Management

What you'll learn in this chapter

- The gap a dependency manager fills that a bare virtual environment (Chapter 12) leaves open: which exact versions belong in it.
- The difference between **declaring** dependencies (ranges you accept) and **locking** them (the exact versions you got) — and why both files exist.
- How to read `hathor-core`'s `pyproject.toml`: metadata, the Python constraint, runtime vs. **dev dependency groups**, version-constraint syntax (`^`, `~`), and a git-sourced dependency.
- What **dependency resolution** is and why it is harder than it sounds.
- What a **build backend** is, and how `pyproject.toml` doubles as packaging config.
- **Why Poetry** and not plain `pip` / `requirements.txt` / `pipenv` — the trade-off.

Chapter 12 gave us the where: an isolated `.venv` with its own `site-packages`. It left a gap — a bare environment has no record of what should fill it or which versions. A `pip install twisted` grabs whatever is newest today, which breaks reproducibility tomorrow. **Poetry**¹ closes that gap: it is the tool `hathor-core` uses to declare its dependencies, resolve and pin them, manage the virtual environment, and package the project. This chapter reads the project's real `pyproject.toml` to make each idea concrete.

It follows the \$5 technology-primer shape, with the "why this and not the alternatives" discussion at the end.

13.1 The problem: a list of packages is not enough

Imagine managing dependencies by hand. You'd keep a list — "we need Twisted, Pydantic, structlog" — and `pip install` them into your environment (Chapter 12). Two problems surface immediately, and they are the problems every dependency manager exists to solve.

Problem 1 — which versions? "We need Twisted" is underspecified. Twisted 22 and Twisted 24 differ; code written against one may break on the other. So you need to record not just which packages but which versions — and not too rigidly, because you also want to accept compatible bug-fix updates without editing the list constantly. You need a way to say "Twisted, somewhere in the 24.7.x range."

Problem 2 — the dependencies of your dependencies. Twisted itself needs other packages (zope.interface, and more), each with its own version requirements, each with their dependencies — the **transitive**² dependency tree. Two of your direct dependencies might demand conflicting versions of a shared deep dependency. Working out a single set of versions that satisfies everyone is **dependency resolution**³, and doing it by hand is infeasible past a handful of packages.

A dependency manager solves both: you declare your direct needs with flexible constraints, and it resolves the entire transitive tree into one consistent set of exact versions, then records that exact set so it can be reproduced. Poetry splits these two jobs across two files.

13.2 Two files: declare in `pyproject.toml`, pin in `poetry.lock`

This split is the heart of Poetry, and of most modern dependency managers. Keep the two roles distinct:

- `pyproject.toml` — what you declare: your direct dependencies, each with a flexible **version constraint** (a range of acceptable versions). This is human-authored and human-readable. It expresses intent: "I want Twisted compatible with 24.7."
- `poetry.lock` — what Poetry resolved: the exact, pinned version of every package, direct and transitive, that satisfies all the constraints together. This is machine-generated, not hand-edited, and it expresses fact: "we are using Twisted exactly 24.7.0, zope.interface exactly X, ..." `hathor-core`'s lock file is ~234 KB — far larger than `pyproject.toml`, because it records the whole resolved tree.

<code>pyproject.toml</code> (you write)		<code>poetry.lock</code> (Poetry generates)
"twisted ~24.7.0"	—resolve→	twisted == 24.7.0
"pydantic ^2.0"		pydantic == 2.x.y
		zope.interface == ... (transitive)
		... every package, exact version ...
intent (ranges)		fact (pinned, reproducible)

Why two files instead of one? Because the two needs conflict. You want **flexibility** when declaring (accept compatible updates) and **exactness** when installing (everyone gets the identical environment). A range can't give reproducibility (it means different

versions on different days); a pinned list can't give flexible updates. So Poetry keeps the range in `pyproject.toml` and the resolved pins in `poetry.lock`. When you run `poetry install`, it reads the lock file and installs those exact versions — so a teammate, a CI runner, and the production Docker image all build a byte-identical environment. When you deliberately want updates, `poetry update` re-resolves the ranges and rewrites the lock. The lock file is committed to git for exactly this reason: it is the reproducibility guarantee.

13.3 Reading `hathor-core`'s `pyproject.toml`

The real file (`pyproject.toml` at the repo root) is the project's dependency manifest. The parts worth reading:

Project metadata and the Python constraint. The package is named `hathor` (`pyproject.toml:16`), and it declares which Python it runs on: `python = ">=3.11,<4"` (`:` 56). That constraint is enforced — Poetry refuses to install into an incompatible interpreter — which is how the platform requirement from Chapter 11 becomes machine-checked rather than a README note.

Runtime dependencies live under `[tool.poetry.dependencies]` (`:` 55). A representative few:

```
twisted      = "~24.7.0"                # the async engine (Ch 16)
pydantic     = "^2.0"                   # validation (Ch 18)
structlog    = "~22.3.0"                # logging (Ch 17)
cryptography = "~42.0.5"                # crypto primitives (Ch 40)
rocksdb      = {git = "https://github.com/hathornetwork/python-rocksdb.git"}
```

Two constraint operators appear, and they mean different things:

- **~ (tilde, "compatible patch")** — `~24.7.0` accepts `>=24.7.0, <24.8.0`: bug-fix updates within 24.7, but not 24.8. Conservative; used where minor changes might risk breakage.
- **^ (caret, "compatible minor")** — `^2.0` accepts `>=2.0, <3.0`: any 2.x, but not 3.0. Looser; used where the project trusts the library's promise not to break within a major version (this is the **semantic versioning**⁴ contract).

The `rocksdb` line is worth noting: instead of a version, it points at a **git URL** — a custom fork maintained by Hathor (`hathornetwork/python-rocksdb`). Dependency managers can pull from sources other than PyPI; here it's because the node needs a specific build of the RocksDB bindings (Chapter 27).

Dependency groups. Development tools — the test runner, type checker, linters — are declared separately under `[tool.poetry.group.dev.dependencies]` (`:` 40): `pytest`, `mypy`, `mypy-zope`, `flake8`, `isort`, `pytest-cov`, `pytest-xdist` (the quality toolchain, Chapter

20). This **grouping** matters: tools needed to develop the node are not needed to run it. When building the production Docker image (Chapter 15), Poetry installs only the main group (`--only=main`), leaving `pytest` and `mypy` out of the shipped image — a smaller, leaner deployment. Separating "what it needs to run" from "what we need to build it" is a real operational win.

Console scripts. The line `hathor-cli = 'hathor_cli.main:main' (:38)` is what creates the `hathor-cli` terminal command, wiring it to the `main()` function (Chapter 21). Poetry installs this as an executable in the environment's `bin/`.

13.4 The build backend: `pyproject.toml` is also packaging config

`pyproject.toml` is not Poetry-specific in origin — it is a Python standard (defined by PEP 518 and PEP 517⁵) for declaring a project's build configuration in one file. The `[build-system]` table (:190) names the **build backend**⁶ — the code that turns the source tree into an installable package:

```
[build-system]
build-backend = "poetry.core.masonry.api"    # pyproject.toml:192
```

A **build backend** is the component that knows how to package a project into the standard distributable formats — a **wheel**⁷ (a pre-built, installable archive) and an **sdist** (a source archive). Here it's Poetry's own backend. This is why one file does double duty: it is both the dependency manifest and the packaging instructions. In the Docker build (Chapter 15), you'll see `poetry build -f wheel` produce a wheel from exactly this config, which is then pip-installed into the final image. The standardization (PEP 517/518) means tools other than Poetry can also read and build the project — the config isn't locked to one tool.

13.5 Why Poetry, and not the alternatives?

Several tools manage Python dependencies. The §5 "why this one" discussion:

vs. plain pip + `requirements.txt`. The oldest approach: a flat text file of packages, installed with `pip install -r`. It works, but it conflates declaration and locking (one file, usually either too loose to reproduce or too pinned to update cleanly), does no real cross-package resolution (pip historically installed greedily and could leave incompatible versions), and doesn't manage the virtual environment or packaging. You end up gluing together pip, `venv`, a separate `setup.py`, and discipline. Poetry unifies all of that.

vs. pipenv. Pipenv pioneered the declare/lock split for applications (`Pipfile` + `Pipfile.lock`) and is the closest predecessor in spirit. Poetry added first-class packaging (the build backend of §13.4) and dependency groups, and converged on the PEP-standard `pyproject.toml` rather than a bespoke `Pipfile`. For a project that is both an application and a publishable package, Poetry's packaging integration is the deciding factor.

vs. newer tools (uv, PDM, Hatch). The ecosystem keeps evolving — faster resolvers and installers have appeared since Poetry. Poetry's advantage for `hathor-core` is maturity and stability: it has been the standard for years, the team's workflow and CI are built around it, and switching a working dependency setup carries risk with little reward. (This is the same "maturity beats novelty" logic that keeps the project on Twisted — Chapter 16.)

The honest summary: Poetry gives one tool for declaring, resolving, locking, environment-managing, and packaging, on top of the PEP-standard `pyproject.toml`. You pay a little — Poetry can be slower than the newest resolvers, and it is one more thing to install — and you get reproducibility and a unified workflow in return. For long-lived infrastructure where "the build must be identical everywhere" is non-negotiable, that trade is firmly worth it.

13.6 Bridge — Poetry across the project

Bridge — Poetry in the project and the stack:

- **It fills the virtual environment of Chapter 12.** Poetry creates/manages the `.venv` and installs the locked set into it — the manager to Chapter 12's container — **Chapter 12**.
- **The Python constraint enforces Chapter 11.** `python = ">=3.11,<4"` (`pyproject.toml:56`) makes the platform requirement machine-checked — **Chapter 11**.
- **Dependency groups shape the Docker image.** `--only=main` excludes dev tools from the production image — **Chapter 15**.
- **Every dependency it pins gets its own chapter.** Twisted (16), structlog (17), Pydantic (18), configargparse (19), cryptography (40), the RocksDB fork (27) — Poetry is the manifest tying them together.
- **The console script is the CLI's front door.** `hathor-cli = 'hathor_cli.main:main' ( :38 )` — **Chapter 21**.
- **The quality tools are declared here.** The dev group lists `pytest/mypy/flake8/isort` — **Chapter 20**.

Recap

Concept	What it is	Where
<code>pyproject.toml</code>	declared deps (ranges) + metadata + build config	repo root
<code>poetry.lock</code>	resolved, pinned exact versions (incl. transitive)	repo root, committed
<code>~</code> / <code>^</code> constraints	patch-compatible / minor-compatible ranges	:57 , :80
dependency groups	runtime vs. dev tooling, installable separately	:40 , :55
git dependency	a dep pulled from a repo, not PyPI	rocksdb :74
build backend	code that packages the project (wheel/sdist)	:192
console script	maps a terminal command to a function	:38
why Poetry	one tool: declare+resolve+lock+env+package	vs pip/pipenv/uv

Poetry is the tool that turns "a list of packages we need" into "the exact, reproducible environment the node runs in everywhere." It splits intent from fact — flexible ranges declared in `pyproject.toml`, exact versions pinned in `poetry.lock` — resolves the whole transitive tree into one consistent set, manages the virtual environment of Chapter 12, separates runtime from development dependencies, and doubles as the project's packaging config via the PEP-standard build backend. It is chosen over pip and its successors for maturity and an all-in-one workflow, accepting some speed cost for guaranteed reproducibility. The next chapter looks at how those many Poetry and tool commands are bundled into memorable shortcuts: **Make**, the project's task runner.

1. Poetry is a Python dependency manager and packaging tool. It declares dependencies in `pyproject.toml`, resolves and pins them in `poetry.lock`, manages a project's virtual environment, and builds distributable packages. ↩
2. A transitive dependency is a dependency of a dependency — a package you don't ask for directly but that something you depend on needs. The full set forms the transitive dependency tree. ↩
3. Dependency resolution is computing a single set of package versions that simultaneously satisfies every direct and transitive constraint. It can be computationally hard and is the core job of a dependency manager. ↩
4. Semantic versioning (semver) is a convention where a version `MAJOR.MINOR.PATCH` signals the kind of change: PATCH = bug fixes, MINOR = backward-compatible features, MAJOR = breaking changes. Constraint operators like `^` rely on this promise. ↩

5. A PEP (Python Enhancement Proposal) is a design document defining a Python standard or feature. PEP 518 and PEP 517 standardized `pyproject.toml` and the build-backend interface, so packaging isn't tied to one tool. ↩
6. A build backend is the component that turns a project's source into installable distributions (a wheel and/or sdist). `pyproject.toml`'s `[build-system]` table names it; Poetry provides its own. ↩
7. A wheel (`.whl`) is Python's standard pre-built package format — an archive that installs without a compile/build step, making installation fast and deterministic. An sdist is the source-archive counterpart. ↩